

Jobsheet 6
Mata Kuliah Praktikum Pemrograman
Berorientasi Objek
Inheritance



Nama : Rachmah Nur Chotimah
NIM : 254107060052
Kelas : SIB 2F

1. PERCOBAAN 1 (extends)

A. Tahapan Percobaan

Hasil Praktikum

```
package Jobsheet_6.Praktikum;

public class Pegawai {
    public Pegawai() {
        System.out.println("Objek dari class Pegawai dibuat");
    }
}
```

```
package Jobsheet_6.Praktikum;

public class Dosen extends Pegawai {
    public Dosen() {
        System.out.println("Objek dari class Dosen dibuat");
    }
}
```

```
package Jobsheet_6.Praktikum;

public class InheritanceDemo {
    public static void main(String[] args) {
        Dosen dosen1 = new Dosen();
    }
}
```

B. PERTANYAAN

1. Pada percobaan 1 diatas, tentukan child class dan parent class!

Jawab: Pada percobaan ini, class Dosen merupakan turunan dari class Pegawai. Yang berarti class Dosen adalah child class dan class Pegawai adalah parent class.

2. Kata kunci apa yang membuat child class dan parent class tersebut memiliki relasi?

Jawab: Hubungan pewarisan tersebut menggunakan kata kunci extends, sehingga Dosen dapat mewarisi atribut atau method dari Pegawai.

3. Berdasarkan hasil yang ditampilkan oleh program, ada berapa constructor yang dieksekusi? Constructor class mana yang lebih dulu dieksekusi?

Jawab: Saat program dijalankan, terdapat 2 constructor yang dieksekusi. Constructor Pegawai sebagai superclass dijalankan terlebih dahulu, kemudian dilanjutkan dengan constructor Dosen sebagai subclass. Hal ini menunjukkan bahwa constructor superclass dipanggil terlebih dahulu sebelum constructor subclass.

2. PERCOBAAN 2 (Pewarisan)

A. Tahapan Percobaan

```
package Jobsheet_6.Praktikum;
```

```

public class Pegawai {
    public String nip;
    public String nama;
    public double gaji;

    public Pegawai() {
        System.out.println("Objek dari class Pegawai dibuat");
    }

    public String getInfo() {
        String info = "";
        info += "NIP      : " + nip + "\n";
        info += "Nama      : " + nama + "\n";
        info += "Gaji       : " + gaji + "\n";

        return info;
    }
}

```

```

public class Dosen extends Pegawai {
    public String nidn;

    public Dosen() {
        System.out.println("Objek dari class Dosen dibuat");
    }
}

```

```

package Jobsheet_6.Praktikum;

public class InheritanceDemo {
    public static void main(String[] args) {
        Dosen dosen1 = new Dosen ();

        dosen1.nama = "Yansy Ayuningtyas";
        dosen1.nip = "34329837";
        dosen1.gaji = 3000000;
        dosen1.nidn = "1989432439";

        System.out.println (dosen1.getInfo());
    }
}

```

B. Pertanyaan

1. Pada percobaan 2 diatas, apakah program dapat berhasil dijalankan ataukah terjadi error?

Jawab: Program berhasil dijalankan tanpa terjadi error.

2. Jika program berhasil dijalankan, mengapa tidak terjadi error pada assignment/pengisian nilai atribut nip, gaji, dan NIDN pada object dosen1 padahal tidak ada deklarasi ketiga atribut tersebut pada class Dosen?

Jawab: Tidak terjadi error karena class Dosen meng-extends class Pegawai. Seluruh atribut bertipe public (nip, nama, gaji) milik Pegawai diwariskan secara otomatis ke class Dosen.

3. Jika program berhasil dijalankan, mengapa tidak terjadi error pada pemanggilan method getInfo() oleh object dosen1 padahal tidak ada deklarasi method getInfo() pada class Dosen?

Jawab: Method getInfo() diwariskan dari parent class (Pegawai) ke child class (Dosen), sehingga objek dosen1 dapat memanggil method tersebut secara langsung.

3. PERCOBAAN 3 (Hak akses)

A. Tahapan Praktikum

1. Modifikasi access level modifier pada atribut gaji menjadi private pada class Pegawai.java

```
public class Pegawai {  
    public String nip;  
    public String nama;  
    private double gaji;  
}
```

2. Run program kemudian amati hasilnya

```
dosen1.gaji = 3000000;    The field Pegawai.gaji is not visible
```

Error tersebut terjadi karena atribut gaji pada class Pegawai memiliki akses private, sehingga tidak dapat diakses langsung dengan class Dosen meskipun Dosen merupakan subclass dari Pegawai.

Hasil Praktikum

```
package Jobsheet_6.Praktikum;  
  
public class Pegawai {  
    public String nip;  
    public String nama;  
    public double gaji;  
  
    public Pegawai() {  
        System.out.println("Objek dari class Pegawai dibuat");  
    }  
  
    public String getInfo() {  
        String info = "";  
        info += "NIP      : " + nip + "\n";  
        info += "Nama     : " + nama + "\n";  
        info += "Gaji      : " + gaji + "\n";  
    }  
}
```

```

        return info;
    }
}

package Jobsheet_6.Praktikum;

public class Dosen extends Pegawai {
    public String nidn;

    public Dosen() {
        System.out.println(gaji);
        System.out.println("Objek dari class Dosen dibuat");
    }
}

package Jobsheet_6.Praktikum;

public class InheritanceDemo {
    public static void main(String[] args) {
        Dosen dosen1 = new Dosen () ;

        dosen1.nama = "Yansy Ayuningtyas";
        dosen1.nip = "34329837";
        dosen1.gaji = 3000000;
        dosen1.nidn = "1989432439";

        System.out.println (dosen1.getInfo());
    }
}

```

B. Pertanyaan

1. Pada langkah 1 di atas, terjadi error karena object dosen1 tidak dapat mengakses atribut gaji. Padahal gaji merupakan atribut Pegawai yang merupakan parent class dari Dosen. Mengapa hal ini dapat terjadi?

Jawab: Terjadi error karena atribut gaji diubah access level modifier-nya menjadi private pada class Pegawai. Atribut private hanya dapat diakses di dalam class itu sendiri dan tidak diwariskan ke subclass.

2. Pada langkah 5, setelah class Pegawai berpindah ke package yang berbeda, class Dosen masih dapat mengakses atribut gaji. Mengapa?

Jawab: Karena modifier gaji diubah menjadi protected. Modifier protected mengizinkan akses dari subclass (Dosen) meskipun berada di package yang berbeda.

3. Berdasarkan percobaan tersebut, bagaimana menentukan atribut dan method yang akan diwariskan oleh parent class ke child class?

Jawab: Atribut dan method yang diwariskan adalah yang memiliki modifier public atau protected. Jika berada dalam package yang sama, modifier default (no modifier) juga diwariskan. Modifier private pernah diwariskan.

4. PERCOBAAN 4 (Super – atribut)

A. Tahapan Praktikum

1. Butlah method getAllInfo() pada class Dosen

```
public String getAllInfo(){
    String info = "";
    info += "NIP      : " + nip + "\n";
    info += "Nama      : " + nama + "\n";
    info += "Gaji       : " + gaji + "\n";
    info += "NIDN      : " + nidn + "\n";

    return info;
}
```

2. Lakukan pemanggilan method getAllInfo() oleh object dosen1 pada class InheritanceDemo.java

```
public static void main(String[] args) {
    Dosen dosen1 = new Dosen();

    dosen1.nama = "Yansy Ayuningtyas";
    dosen1.nip = "34329837";
    dosen1.gaji = 3000000;
    dosen1.nidn = "1989432439";

    System.out.println(dosen1.getAllInfo());
}
```

3. Run program kemudian amati hasilnya

```
Objek dari class Pegawai dibuat
Objek dari class Dosen dibuat
NIP   : 34329837
Nama  : Yansy Ayuningtyas
Gaji  : 3000000.0
NIDN  : 1989432439
```

4. Lakukan modifikasi method getAllInfo() pada class Dosen

```
public String getAllInfo(){
    String info = "";
    info += "NIP      : " + this.nip + "\n";
    info += "Nama      : " + this.nama + "\n";
    info += "Gaji       : " + this.gaji + "\n";
    info += "NIDN      : " + this.nidn + "\n";

    return info;
}
```

5. Run program kemudian bandingkan hasilnya dengan langkah no 2.

```
Objek dari class Pegawai dibuat
Objek dari class Dosen dibuat
NIP   : 34329837
Nama  : Yansy Ayuningtyas
Gaji  : 3000000.0
NIDN  : 1989432439
```

Perbandingan: Kedua kode tersebut menghasilkan output data NIP, Nama, Gaji, dan NIDN yang identik. Hal ini terjadi karena pemanggilan variabel secara langsung (seperti nip) maupun menggunakan *keyword* (seperti this.nip) sama-

sama merujuk pada nilai atribut dari objek dosen1 (objek yang sedang aktif) yang diwariskan dari *parent class*.

6. Lakukan modifikasi method `getAllInfo()` pada class Dosen Kembali

```
public String getAllInfo(){
    String info = "";
    info += "NIP      : " + super.nip + "\n";
    info += "Nama       : " + super.nama + "\n";
    info += "Gaji        : " + super.gaji + "\n";
    info += "NIDN       : " + super.nidn + "\n";

    return info;
}
```

7. Run program kemudian bandingkan hasilnya dengan program pada no 1 dan no 4.

```
public String getAllInfo() {
    String info = "";
    info += "NIP      : " + super.nip + "\n";
    info += "Nama       : " + super.nama + "\n";
    info += "Gaji        : " + super.gaji + "\n";
    info += "NIDN       : " + super.nidn + "\n";
}
```

nidn cannot be resolved or is not a field

Perbandingan: Penyebab error "nidn cannot be resolved or is not a field" pada baris `super.nidn` adalah karena kata kunci `super` hanya digunakan untuk mengakses atribut atau method yang ada di dalam parent class (kelas induk). Atribut `nidn` dideklarasikan di dalam child class (Dosen), bukan di dalam parent class (Pegawai). Oleh karena itu, Java tidak mengenal `super.nidn`, dan untuk mengakses `nidn` harus menggunakan `this.nidn`

8. Lakukan modifikasi method `getAllInfo()` pada class Dosen kembali

```
public String getAllInfo(){
    String info = "";
    info += "NIP      : " + super.nip + "\n";
    info += "Nama       : " + super.nama + "\n";
    info += "Gaji        : " + super.gaji + "\n";
    info += "NIDN       : " + this.nidn + "\n";

    return info;
}
```

9. Run program kemudian bandingkan hasilnya dengan program pada no 2 dan no 4.

```
Objek dari class Pegawai dibuat
Objek dari class Dosen dibuat
NIP      : 34329837
Nama     : Yansy Ayuningtyas
Gaji     : 3000000.0
NIDN     : 1989432439
```

Perbandingan: Hasil eksekusi program ini dengan langkah 2 dan langkah 4 sama persis dan tidak memiliki perbedaan. Kedua program tersebut menghasilkan urutan teks pembuatan objek beserta data NIP, Nama, Gaji, dan NIDN yang identik. Kesamaan hasil ini terjadi karena pemanggilan atribut secara langsung (seperti `nip`) maupun menggunakan awalan (seperti `this.nip`) sama-sama merujuk pada variabel dari current object atau instance objek yang sedang aktif (dalam hal ini, objek `dosen1`).

B. Pertanyaan

1. Apakah terdapat perbedaan hasil nama, nip, dan gaji yang ditampilkan pada program 1, 4, dan 8? Mengapa?

Jawab: Tidak ada perbedaan hasil output antara program 1, 4, dan 8. Hal ini karena nip, nama, dan gaji diwariskan dari parent class, sehingga pemanggilan tanpa keyword, dengan this., maupun dengan super. akan merujuk ke data/atribut yang sama.

2. Mengapa error terjadi pada program no 6?

Jawab: Terjadi error pada program no 6 karena baris `super.nidn` mencoba mengakses atribut `nidn` menggunakan kata kunci `super`. Padahal `nidn` adalah atribut asli milik class Dosen (child class), bukan milik class Pegawai (parent class).

5. PERCOBAAN 5 (super & overriding)

A. Tahapan Praktikum

Hasil Praktikum

```
Objek dari class Pegawai dibuat
Objek dari class Dosen dibuat
NIP   : 34329837
Nama  : Yansy Ayuningtyas
Gaji  : 3000000.0
NIDN  : 1989432439
```

```
public String getInfo() {
    return "NIDN : " + this.nidn + "\n";
}

public String getAllInfo() {
    String info = super.getInfo();
    info += this.getInfo();

    return info;
}
```

B. PERTANYAAN

1. Apakah ada perbedaan method `getInfo()` yang diakses pada langkah 1, 2, dan 3?

Jawab: Tidak ada perbedaan hasil antara langkah 1, 2, dan 3. Ketiganya mengeksekusi method `getInfo()` milik parent class (Pegawai).

2. Apakah ada perbedaan method `super.getInfo()` dan `this.getInfo()` yang dipanggil dalam method `getAllInfo()` pada langkah 4? Jelaskan!

Jawab: Terdapat perbedaan dalam pemanggilan method `getInfo()`. `super.getInfo()` digunakan untuk memanggil method milik parent class, yaitu Pegawai. Sedangkan `this.getInfo()` memanggil method milik class saat ini, yaitu Dosen, yang sudah melakukan *override* terhadap method tersebut.

3. Pada method apakah terjadi overriding? Jelaskan!

Jawab: Overriding terjadi pada method **getInfo()**. Class Dosen mendefinisikan ulang method getInfo() yang sebelumnya sudah ada pada class Pegawai dengan implementasi yang berbeda.

6. PERCOBAAN 6 (overloading)

A. Tahapan Praktikum

Hasil Praktikum

```
public class Dosen extends Pegawai {
    public String nidn;

    // Constructor tanpa parameter
    public Dosen(String nip, String nama, double gaji, String nidn) {
        System.out.println("Objek dari class Dosen dibuat dengan
constructor berparameter");
    }

    public String getInfo() {
        return "NIDN : " + this.nidn + "\n";
    }

    public String getAllInfo() {
        String info = super.getInfo();
        info += this.getInfo();

        return info;
    }
}
```

```
public class InheritanceDemo {
    public static void main(String[] args) {
        Dosen dosen2 = new Dosen("34329837", "Yansy Ayuningtyas",
3000000, "1989432439");
        System.out.println(dosen2.getAllInfo());
    }
}
```

```
Objek dari class Pegawai dibuat
Objek dari class Dosen dibuat dengan constructor berparameter
NIP   : null
Nama  : null
Gaji  : 0.0
NIDN  : null
```

B. PERTANYAAN

1. Bagaimana hasil nilai nip, nama, gaji, dan nidn yang ditampilkan pada langkah 2? Mengapa demikian?

Jawab: Nilai nip, nama, gaji, dan nidn tampil sebagai null atau 0.0. Hal ini terjadi karena kode di dalam constructor berparameter pada langkah 1 hanya mencetak

kalimat dan tidak melakukan penugasan (*assignment*) nilai parameter ke atribut objek.

2. Jelaskan apakah constructor tanpa parameter dan constructor class Dosen yang dibuat pada langkah 1 memiliki signature yang sama?

Jawab: Tidak sama. Kedua constructor memiliki *signature* yang berbeda karena jumlah dan tipe parameternya berbeda (constructor 1 tanpa parameter, sedangkan constructor 2 memiliki 4 parameter).

3. Konsep apa dalam OOP yang membolehkan suatu class memiliki constructor atau method dengan nama yang sama dan signature yang berbeda pada satu class?

Jawab: Konsep OOP tersebut disebut Overloading, yaitu ketika dalam satu class terdapat constructor atau method dengan nama yang sama tetapi memiliki parameter (signature) yang berbeda. Hal ini memungkinkan method atau constructor digunakan dengan kebutuhan yang berbeda.

7. PERCOBAAN 7 (super - constructor)

A. TAHAPAN PERCOBAAN

Hasul Percobaan

```
public class Pegawai {
    public String nip;
    public String nama;
    public double gaji;

    // Constructor tanpa parameter
    public Pegawai() {
        System.out.println("Objek dari class Pegawai dibuat");
    }

    // Constructor dengan parameter
    public Pegawai(String nip, String nama, double gaji) {
        this.nip = nip;
        this.nama = nama;
        this.gaji = gaji;
    }

    public String getInfo() {
        String info = "";
        info += "NIP : " + nip + "\n";
        info += "Nama : " + nama + "\n";
        info += "Gaji : " + gaji + "\n";
        return info;
    }
}

public class Dosen extends Pegawai {
```

```

    public String nidn;

    // Constructor tanpa parameter
    public Dosen() {
        System.out.println("Objek dari class Dosen dibuat");
    }

    // Constructor dengan parameter
    public Dosen(String nip, String nama, double gaji, String nidn) {
        super(nip, nama, gaji); // Memanggil constructor dari class
        Pegawai
        this.nidn = nidn;
    }

    public String getAllInfo() {
        String info = super.getInfo();
        info += "NIDN : " + nidn + "\n";
        return info;
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Dosen dosen2 = new Dosen("34329837", "Yansy Ayuningtyas",
3000000, "1989432439");
        System.out.println(dosen2.getAllInfo());
    }
}

```

<pre> NIP : 34329837 Nama : Yansy Ayuningtyas Gaji : 3000000.0 NIDN : 1989432439 </pre>	
--	--

B. PERTANYAAN

- Apakah terdapat perbedaan hasil pada langkah 1 dan 2? Jelaskan!
Jawab: Tidak ada perbedaan hasil. Penulisan `this.nip` dan `super.nip` merujuk ke lokasi atribut yang sama karena nip diwariskan dari parent class.
- Apakah terdapat perbedaan hasil pada langkah 2 dan 3? Jelaskan!
Jawab: Tidak ada perbedaan hasil. Pemanggilan `super()` tanpa parameter dilakukan secara implisit oleh Java jika tidak dituliskan.
- Mengapa terjadi error pada langkah 4?
Jawab: Terjadi error karena di class `Dosen` terdapat constructor yang memanggil `super()` (constructor default tanpa parameter), sedangkan constructor tanpa parameter di class `Pegawai` sudah dihapus/di-comment.
- Apa perbedaan `super()` yang dipanggil pada langkah 3 dan 6?

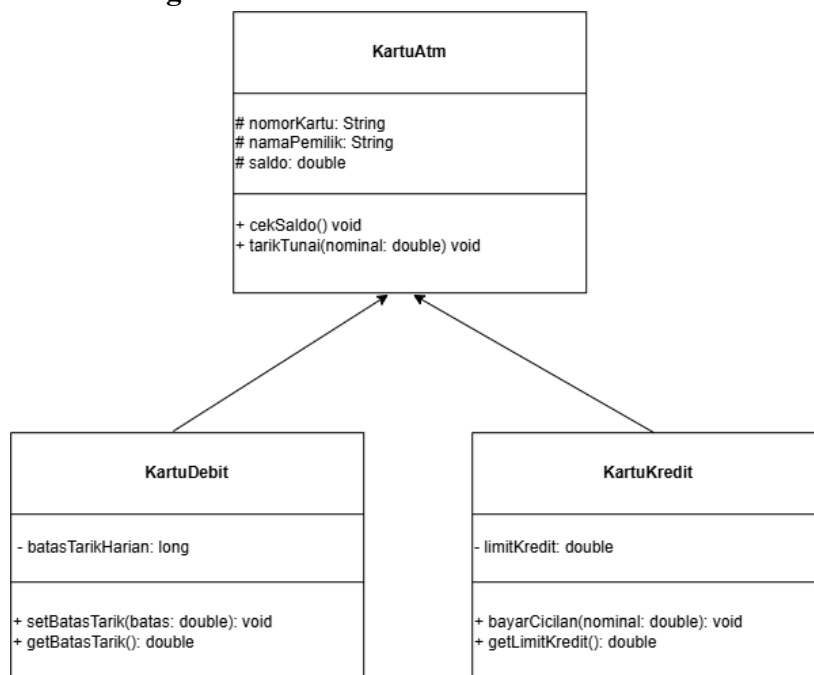
Jawab: Perbedaannya terletak pada constructor yang dipanggil. `super()` digunakan untuk memanggil constructor default milik Pegawai tanpa parameter, sedangkan `super(nip, nama, gaji)` digunakan untuk memanggil constructor Pegawai yang memiliki parameter.

5. Mengapa terjadi error pada langkah 5?

Jawab: Terjadi error karena pemanggilan `super(...)` harus menjadi instruksi/baris pertama di dalam tubuh constructor. Pada langkah 5, `super(...)` diletakkan di baris kedua setelah `this.nidn = nidn;`.

8. TUGAS

Class Diagram Hasil Tugas Teori PBO



Hasil Implementasi

Parent

```
// parent
public class KartuAtm {
    // protected (#) agar atribut ini hanya bisa diakses dan diwariskan
    kepada child
    protected String nomorKartu;
    protected String namaPemilik;
    protected double saldo;

    // Constructor
    public KartuAtm() { // kosongan
    }
    public KartuAtm(String nomorKartu, String namaPemilik, double saldo){
        this.nomorKartu = nomorKartu;
        this.namaPemilik = namaPemilik;
    }
}
```

```

        this.saldo = saldo;
    }

    // method
    public void cekSaldo(){
        System.out.println("Saldo " + namaPemilik + " saat ini: Rp" +
saldo);
    }

    public void tarikTunai(double nominal) {
        if (saldo >= nominal) {
            saldo -= nominal;
            System.out.println("Berhasil menarik Rp" + nominal + ". Saldo
saldo: Rp" + saldo);
        } else {
            System.out.println("Maaf, saldo tidak mencukupi!");
        }
    }
}

```

Child 1

```

public class KartuDebit extends KartuAtm {
    // Modifer private (-) agar atribut tidak dilihat di class lain
    private long batasTarikHarian;

    // Contructor
    public KartuDebit(String nomorKartu, String namaPemilik, double
saldo, long batasTarikHarian) {
        super(nomorKartu, namaPemilik, saldo);
        this.batasTarikHarian = batasTarikHarian;
    }

    // method
    public void setBatasTarik(long batas) {
        this.batasTarikHarian = batas;
        System.out.println("Batas tarik harian diubah menjadi: Rp" +
batas);
    }

    public long getBatasTarik() {
        return batasTarikHarian;
    }
}

```

Child 2

```

public class KartuKredit extends KartuAtm {
    // Modifier private (-)
    private double limitKredit;

    // Konstruktor memanggil super() dari parent
}

```

```

    public KartuKredit(String nomorKartu, String namaPemilik, double
saldo, double limitKredit) {
        super(nomorKartu, namaPemilik, saldo);
        this.limitKredit = limitKredit;
    }

    public void bayarCicilan(double nominal) {
        saldo += nominal; // Asumsi bayar cicilan mengembalikan saldo
limit yang terpakai
        System.out.println("Berhasil bayar cicilan Rp" + nominal + ".
Saldo saat ini: Rp" + saldo);
    }

    public double getLimitKredit() {
        return limitKredit;
    }
}

```

```

public class DemoKartu {
    public static void main(String[] args) {
        System.out.println("=== TEST KARTU DEBIT ===");
        KartuDebit debitRahma = new KartuDebit("1234-5678", "Rahma",
5000000, 10000000);
        debitRahma.cekSaldo(); // Memanggil method warisan dari parent
debitRahma.tarikTunai(500000); // Memanggil method warisan dari
parent
        System.out.println("Batas tarik harian: Rp" +
debitRahma.getBatasTarik()); // Method spesifik class anak
        // modifikasi nilai
        debitRahma.setBatasTarik(15000000);
        System.out.println("\n=== SETELAH DIMODIFIKASI ===");
        System.out.println("Batas tarik harian: Rp" +
debitRahma.getBatasTarik());

        System.out.println("\n=== TEST KARTU KREDIT ===");
        KartuKredit kreditRahma = new KartuKredit("8765-4321", "Rahma",
2000000, 15000000);
        kreditRahma.cekSaldo(); // Memanggil method warisan dari parent
kreditRahma.bayarCicilan(1000000); // Method spesifik class anak
    }
}

```

```
041c4f655e9de42e71159265b800074e (reana@:java) (juc_ws) (PrakPBO_205a)
=== TEST KARTU DEBIT ===
Saldo Rahma saat ini: Rp5000000.0
Berhasil menarik Rp500000.0. Saldo saldo: Rp4500000.0
Batas tarik harian: Rp10000000
Batas tarik harian diubah menjadi: Rp15000000

=== SETELAH DIMODIFIKASI ===
Batas tarik harian: Rp15000000

=== TEST KARTU KREDIT ===
Saldo Rahma saat ini: Rp2000000.0
Berhasil bayar cicilan Rp1000000.0. Saldo saat ini: Rp3000000.0
```

Link Github : https://github.com/lovvovala/PrakPBO/tree/main/Jobsheet_6